Homework 2

EK505 - Prof. Tron

2026-10-08

Goal The goal of this assignment is to implement and test a gradient descent method for position-only inverse kinematics. You will apply the method to the SCARA robot from Homework 1 and to the Panda arm from the Robotics Toolbox, and compare your implementation with the inverse kinematics routines provided by the Robotics Toolbox.

Problem 1: Gradient descent inverse kinematics

Question report **1.1.** quiz As seen in a previous in-class activity, the Robotics Toolbox method `.jacob0(_)` returns the geometric Jacobian of a robot. For position-only inverse kinematics, we will use the first three rows of this Jacobian:

```
Jp = robot.jacob0(q)[:3, :]
```
(1)

Explain the meaning of the different rows, and why, in this assignment, we only use the first three.

Question code **1.1.** Implement a reusable gradient descent solver for position-only inverse kinematics. The function should minimize the objective

$$\frac{1}{2}\|p(q) - p_{\text{des}}\|^2, \tag{2}$$

using the update seen in class, where the Jacobian J_p is given by (1).

> File name: `ek505_ik.py`
> Function name: `ikine_position_gradient`
> Description: Compute a joint configuration whose end-effector position is close to a desired target position by using gradient descent on the position error.
> Input arguments
>
> - `robot`: robot model, compatible with the Robotics Toolbox interface.
> - `p_des` (dim. [3 ×], type `np.array`): desired end-effector position, expressed in the base frame.
> - `q0` (length `robot.n`, type `array-like`): initial joint configuration.
> - `step_size`: gradient descent step size. The default value should be `0.1`.
> - `tol`: convergence tolerance on the position error norm. The default value should be `1e-3`, corresponding to 1 mm.

- `max_iters`: maximum number of gradient descent iterations. The default value should be `1000`.
- `joint_limits`: Boolean variable indicating whether joint limits should be enforced. The default value should be `True`.

Output arguments

- `q_sol` (dim. [1 × `robot.n`], type `np.array`): final joint configuration.
- `flag_status` (type `Bool`): `True` if the final error is below `tol`.
- `history_cost` (dim. [`nb_iterations` × 1], type `np.array`): array of values of the cost at each iteration.
- `history_joint` (dim. [`nb_iterations` × `robot.n`], type `np.array`): array of the joint values at each iteration.

In the above, `nb_iterations` indicates the number of iterations performed.

Your function should initialize the array `q_sol` with the initial guess `q0` and an empty array `history`, and then, for each iteration:

- compute the current value for the IK cost (distance between current and desired position), and store it in `history_cost`; store the current values for the joints in `history_joint`
- compute all the quantities necessary for implementing one gradient descent update;
- update the configuration using the gradient descent update;
- enforce joint limits;

The iterations should stop when the position error norm is below `tol`, or when the maximum number of iterations is reached;

Question **Pre-LLM** **1.1.** **quiz** Write the pseudocode for the function `ikine_position_gradient(_)`. In particular, explain how the implementation of Question code 1.1 handles joint limits. In particular, explain:

1) what are *all* the quantities necessary to compute the gradient descent update, and how they can be obtained;

2) how to obtain the joint limits are obtained from the robot model;

3) how the gradient descent update is modified or projected to enforce the aforementioned limits.

Question **Pre-LLM** **1.2.** If you are not already familiar, read the documentation of `open(f)`unction (`https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-`) and the `pickle` module (`https://docs.python.org/3/library/pickle.html`), in particular of the `pickle.dump(_)` (`https://docs.python.org/3/library/pickle.html#pickle.dump`) and `pickle.load(_)` (`https://docs.python.org/3/library/pickle.html#pickle.load`). Give an example of a script that

1) opens a file for writing,

2) saves the contents of the dictionary `data={'field1':1,'field2':[1,1,1]}`,

3) opens the same file for reading,

4) loads the dictionary back from the file.

Question code 1.2. Implement a function to compare on different robots your gradient descent inverse kinematics solver with one of the inverse kinematics methods provided by the Robotics Toolbox.

> File name: `ek505_ik.py`
>
> Function name: `ikine_comparison_test`
>
> Description: Run position-only inverse kinematics on a fixed set of target positions using both the gradient descent solver from Question code 1.1 and the Robotics Toolbox inverse kinematics method `.ikine_LM()`. Save the results to a `results_filename` pickle file.
>
> Input arguments
>
> - `robot`: robot model.
> - `targets` (dim. [`nb_targets` × 3], type `np.array`): desired end-effector positions.
> - `q0` (length `robot.n`, type `array-like`): initial joint configuration used for both methods.
> - `results_filename` (type `String`): name of the pickle file where the results should be saved.
> - `step_size`: gradient descent step size. The default value should be `0.1`.
> - `tol`: convergence tolerance. The default value should be `1e-3`.
> - `max_iters`: maximum number of gradient descent iterations. The default value should be `1000`.
>
> Output arguments
>
> - The function does not need to return anything, but it should save all results as a dictionary `data` in a `pickle` file with name `results_filename`.

For the Robotics Toolbox method, use a position-only mask (please see the documentation for `.ikine_LM()` for details):

```
mask = [1, 1, 1, 0, 0, 0]
```

For example, you may use `.ikine_LM()` with a desired pose whose translational part is the target position. Since the orientation is masked out, the desired orientation is not important.

The function should use the same target positions and the same initial configuration for both methods. For each target, record:

- the target position;
- the initial configuration;
- the final configuration from gradient descent;

- the final configuration from the Robotics Toolbox method;
- the success/failure flag for each method;
- the final position error for each method;
- the gradient descent histories.

The saved pickle file will be used later for plotting and visualization.

Question report **1.2.** Estimate, using geometric intuition, a simple region of task space (e.g., box, cylinder, or sphere) where position inverse kinematics is expected to succeed for each of the following robots:

- the SCARA robot from Homework 1;
- the Panda robot from the Robotics Toolbox (available through the class `rtb.models.Panda()`).

Your estimates do not need to be exact. Use simple geometric reasoning based on link lengths, joint types, and joint limits. Motivate your choice in the report.

Question report **1.3.** Run the inverse kinematics comparison experiments for the SCARA and Panda robots.

First, generate `nb_targets = 20` target positions for each robot (e.g., using `np.random.rand()`) in the estimated task-space regions from Question report 1.2. Then, use the function from Question code 1.2 to run the comparison experiments for:

- the SCARA robot from Homework 1;
- the Panda robot from the Robotics Toolbox.

Use a convergence tolerance of `tol = 1e-3` meters for both methods. Use the configuration with all-zeros angles as an initial guess.

Save the full experiment results to separate pickle files: `scara_ik_results.pkl` and `panda_ik_results.pkl`.

For each robot, summarize the success rates in a 2×2 table, where each entry is a percentage of the 20 tested target positions.

	RTB succeeds	RTB fails
Gradient succeeds	both succeed (green)	gradient only (blue)
Gradient fails	RTB only (orange)	both fail (red)

The colors are used later in Questions video 1.1 and video 1.2.

For each robot, also report the average final position error for the targets where both methods succeed:

Method	Average final position error
Gradient	
RTB	

Question report **1.4.** quiz Discuss the differences between the inverse kinematics solutions obtained by your gradient descent method and by the Robotics Toolbox method. In particular:

- explain why two inverse kinematics solvers may return different joint configurations for the same target position;

- discuss how redundancy in the Panda arm affects the solution, especially compared with the SCARA robot;
- discuss how using multiple random initial configurations could affect the success rate, the final joint configuration, and the final position error.

Question report 1.5. Plot convergence histories using the saved pickle files from Question report 1.3, `scara_ik_results.pkl`, `panda_ik_results.pkl`. Choose one representative target from each file. For each robot, compare the gradient-descent and RTB results by plotting:

- position error norm versus iteration;
- each joint value versus iteration.

Use the saved gradient descent history from the pickle file. Since the Robotics Toolbox method does not provide an iteration history, show only its final solution as horizontal reference lines in the plots.

Question video 1.1. Create a Swift visualization for the SCARA robot using the results that were saved in the file `scara_ik_results.pkl` and the same target used for Question report 1.5. Create a video showing the SCARA robot moving through the gradient descent inverse kinematics iterations.

In the same Swift scene, visualize all `nb_targets` SCARA target positions using small spheres. Color the spheres according to the same categories used in the table from Question report 1.3:

- green: both gradient descent and RTB succeed;
- blue: gradient descent succeeds and RTB fails;
- orange: gradient descent fails and RTB succeeds;
- red: both methods fail.

🔎 Spheres can be added to the simulation using the command

```
env.add(sg.Sphere(0.05, pose = sm.SE3(.5, .5, .5), color=[1,0,0,1]))
```

where `env` is the variable with the Swift environment launched, `sm` is the `spatialmath` module, `sg` is the `spatialgeometry` module, the arguments of `sm.SE3(_)` contain a translation in world coordinates, and the `color` is specified with a RGBA quadruplet (red, in the example).

Submit a short screen recording.

Question video 1.2. Repeat the same steps as in Question video 1.1 but for the Panda robot, using the results for `panda_ik_results.pkl` and including the visualization of the target positions using small colored spheres. Submit another short screen recording.